Name: Solomon Getu
Student ID: CTC-6335-26
Department: Cyber Secuirty

Given Task: Write write up for API testing

CONTENTS

1. Introduction
2. Definition of API testing
3. API testing Labs
4. Conclusion

Monitor: Gedion Desalegn
Date: Aug 16, 2026 G.C

1. Introduction

- What is an API?

- **APIs** stands for **Application Programming Interfaces**.
- They enable software systems, and applications to communicate and share data.
- API testing is important as vulnerabilities in APIs may undermine core aspects of a website's *confidentiality, integrity, and availability*.

For example:

User → Website → API → Database

↓
Response

- When you use a website, the front-end may send requests to an API to retrieve or modify information.
- **Why is API testing important?**
- API testing is important because vulnerabilities in APIs can affect the:

  - **Confidentiality** :- protecting information from unauthorized access.
  - **Integrity** :- preventing unauthorized modification of data.
  - **Availability** :- ensuring systems and services remain available.

- The source also points out that dynamic websites are composed of APIs, so traditional web vulnerabilities can also be relevant to API testing.

In short, my write up includes such kind of concepts, and some practical Labs.

2. Definition of API Testing

- **API testing** is the process of examining an API to determine whether it behaves as expected and whether it contains security vulnerabilities.

- In security testing, this includes examining:

- ✓ API endpoints
- ✓ HTTP methods
- ✓ Parameters
- ✓ Authentication Input data
- ✓ Content types
- ✓ Error messages
- ✓ Hidden endpoints
- ✓ Hidden parameters
- ✓ API documentation
- ✓ Access controls

- The material focuses particularly on ***RESTful* and *JSON APIs,*** as well as **server-side parameter pollution**.

3. API testing Labs

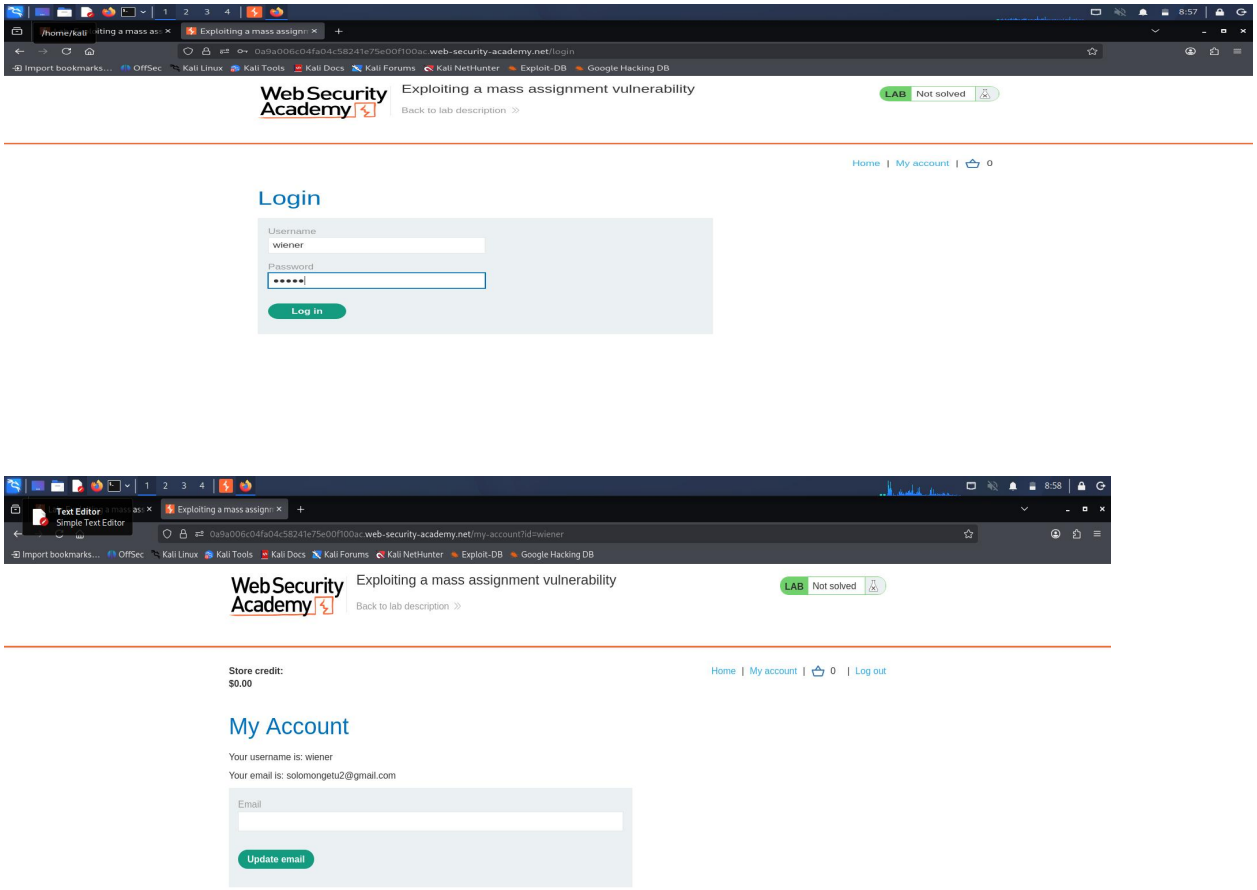
1. Lab1- Exploiting API end point using API documentation
https://portswigger.net/web-security/api-testing/lab-exploiting-api-endpoint-using-documentation

First of all, what are End point, and API documentation

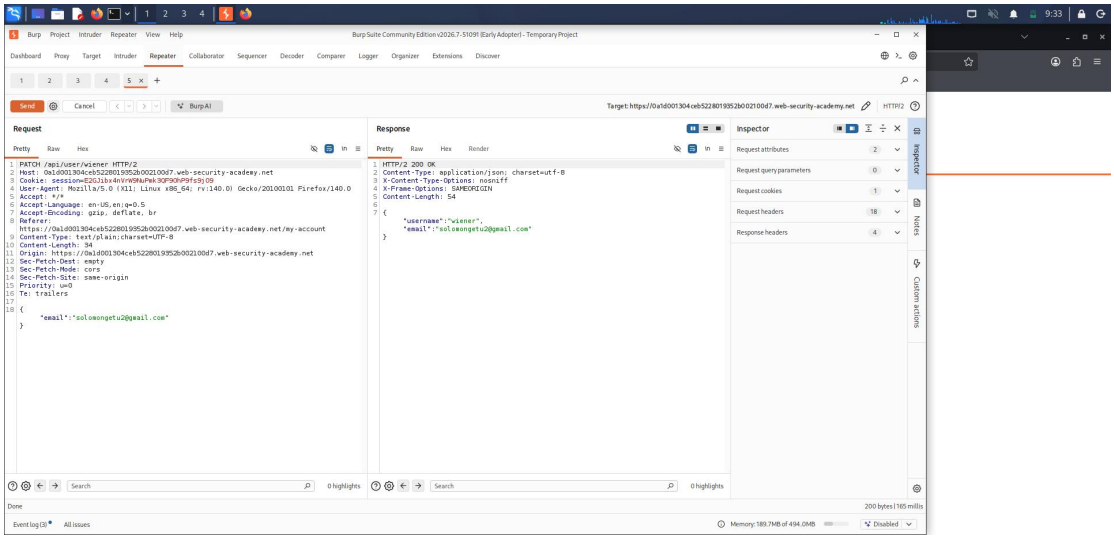
- **API Documentation**, as I mention previously, it is simply a guide that explains how to use an API.
- **API End point** is a specific URL where an API receives a request and provides a response. Each URL, and HTTP methods like GET, POST,PUT, PATCH, and DELETE represents an endpoint.

My Walkthrough In LAB1

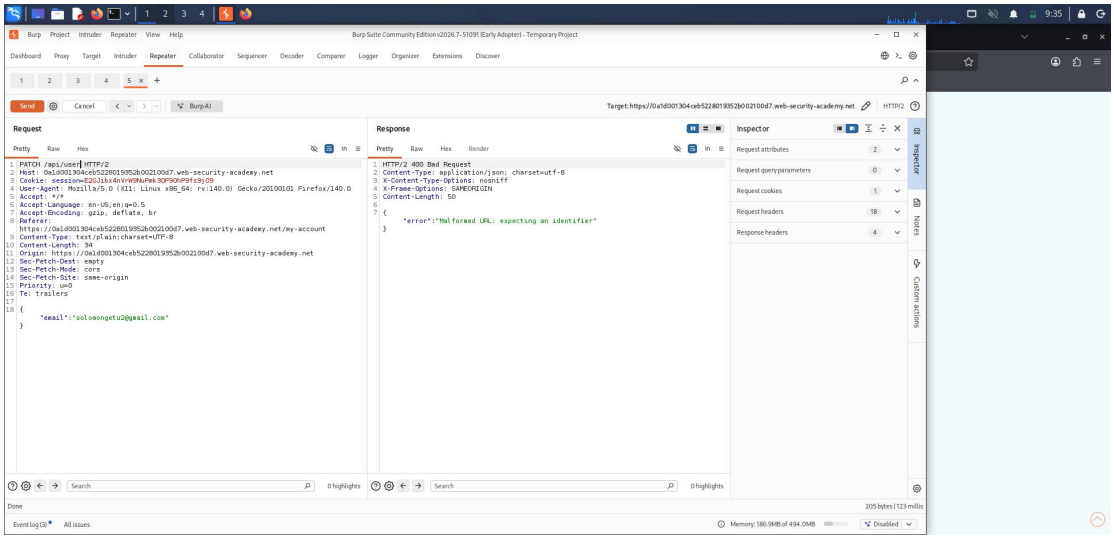
1. In Burp's browser, I logged in to the application using the credentials wiener:peter and updated my email address.



2. I went to **Proxy > HTTP history**. Right-clicked the ***PATCH /api/user/wiener*** request and selected **Send to Repeater**.

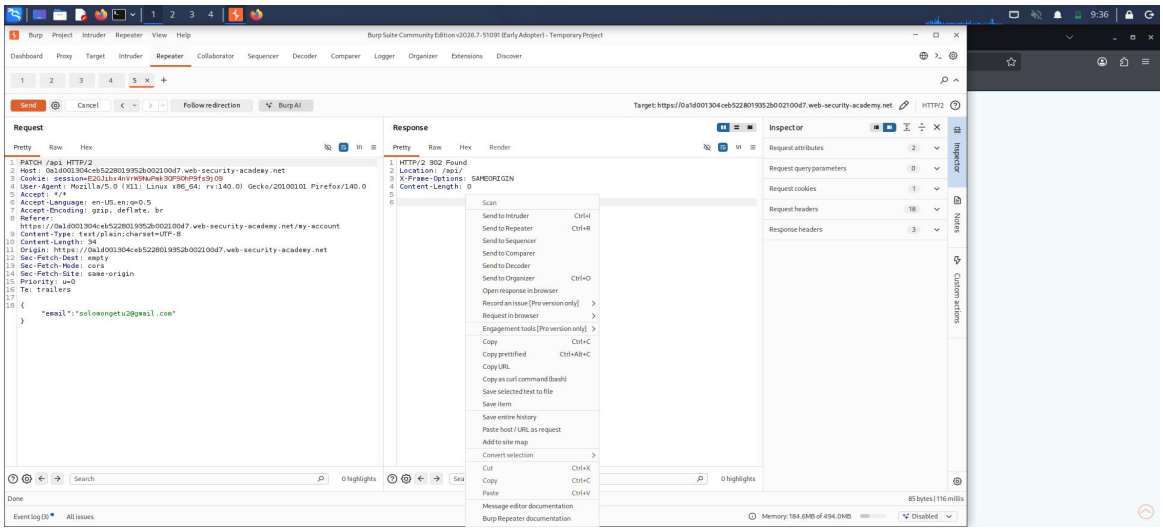


3.I went to the **Repeater** tab and sent the ***PATCH /api/user/wiener*** request. Noticed that it retrieved credentials for the user wiener.



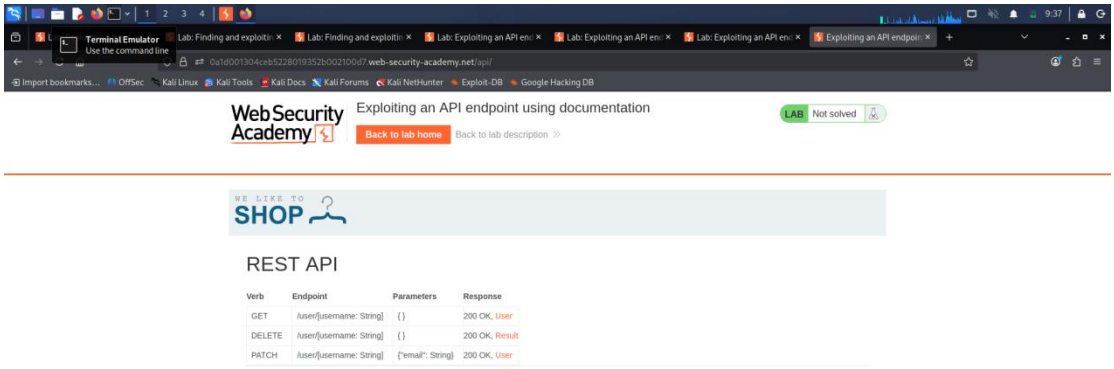
4. I removed **/wiener** from the path, so the endpoint became **/api/user**. I sent the request. Noticed that it returned an error because there was no user identifier.

5. Then I removed **/user** from the path, so the endpoint became /api. I sent the request. Noticed that it retrieved the API documentation.

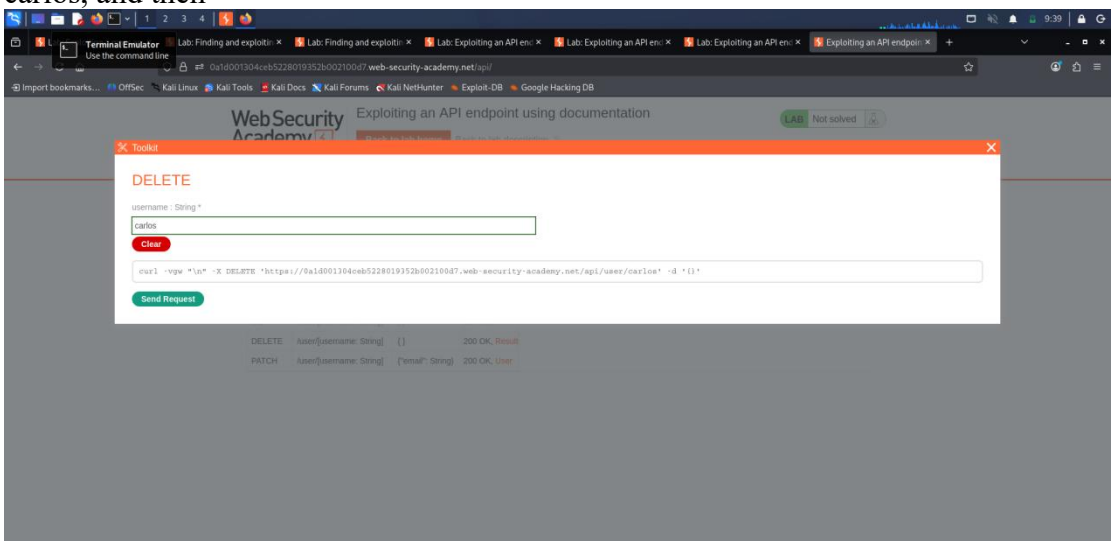


5.I right-clicked the response and selected **Show response in browser,** and then copied the URL.

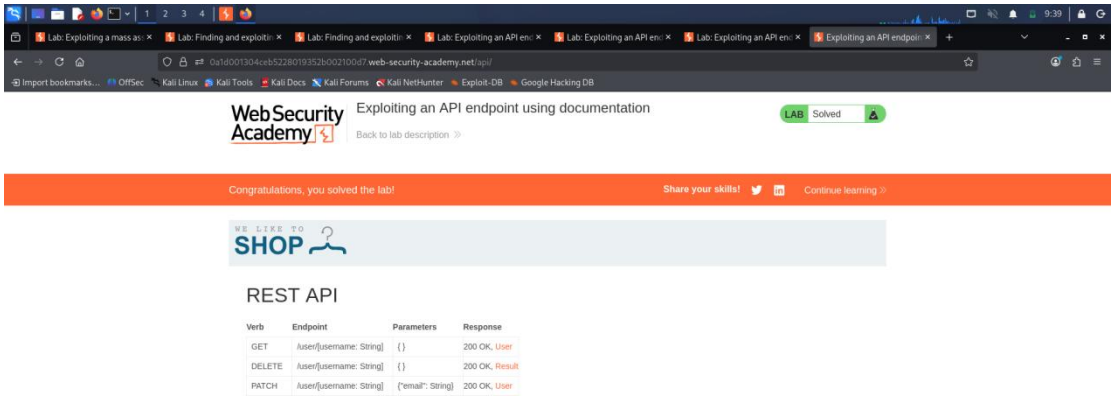
6. I pasted the URL into Burp's browser to access the documentation. Noticed that the documentation was interactive.



7. Then I deleted Carlos and solved the lab, by clicking the **DELETE** row, entered carlos, and then



clicked **Send request**.



To sum up, the **/api endpoint** returned the API documentation. This showed that **API documentation** can expose information about available API endpoints and the operations that can be performed on them. I found that the documentation was interactive, meaning that API requests could be made directly through the documentation. I used this information to identify the DELETE operation and successfully delete the Carlos user.

2. LAB2 Exploiting Unused API endpoint

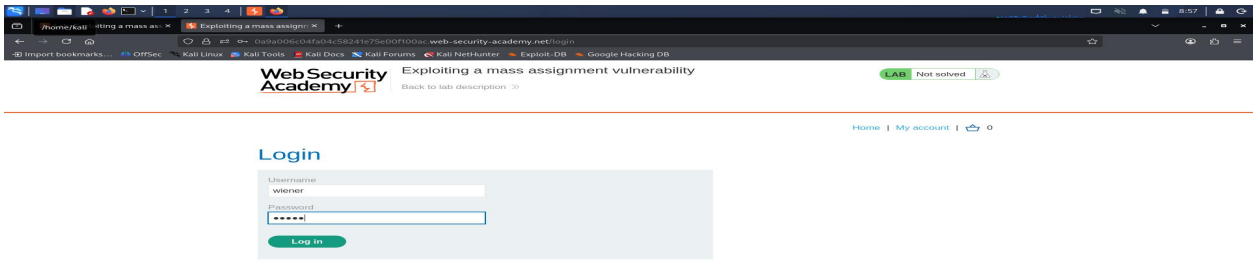
https://portswigger.net/web-security/api-testing/lab-exploiting-unused-api-endpoint

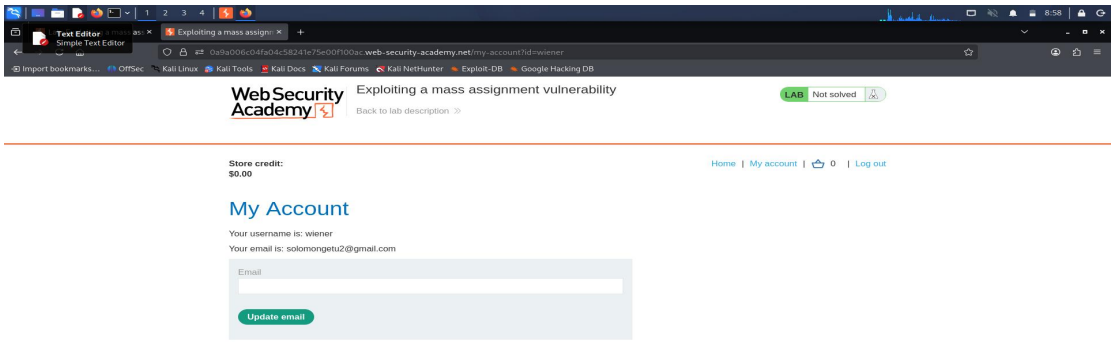
What the lab is teaching

- An API may have an endpoint that exists on the server but is **not used by the website's front-end**. So, even if we don't see a particular function in the normal website interface, the API might still expose it.

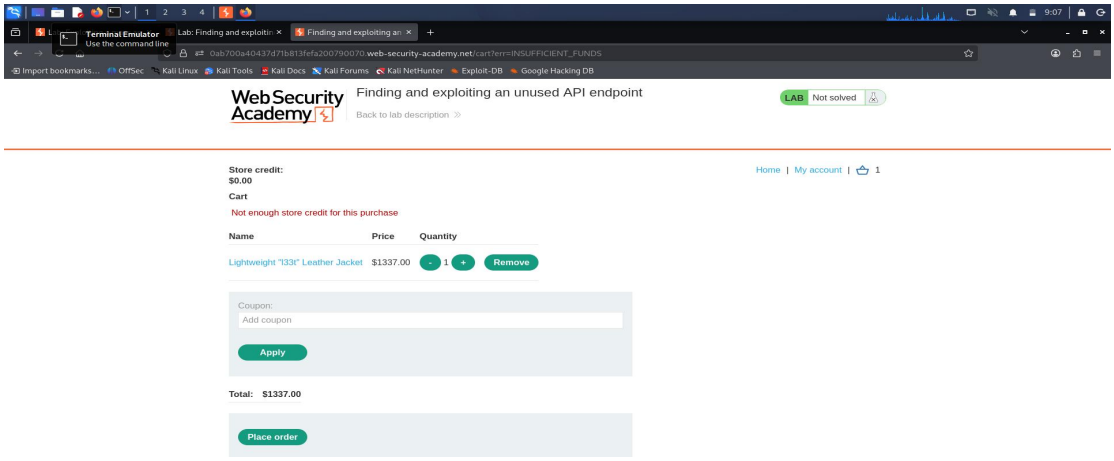
My Walkthrough In LAB2

1. In Burp's browser, I logged in to the application using the credentials wiener:peter.

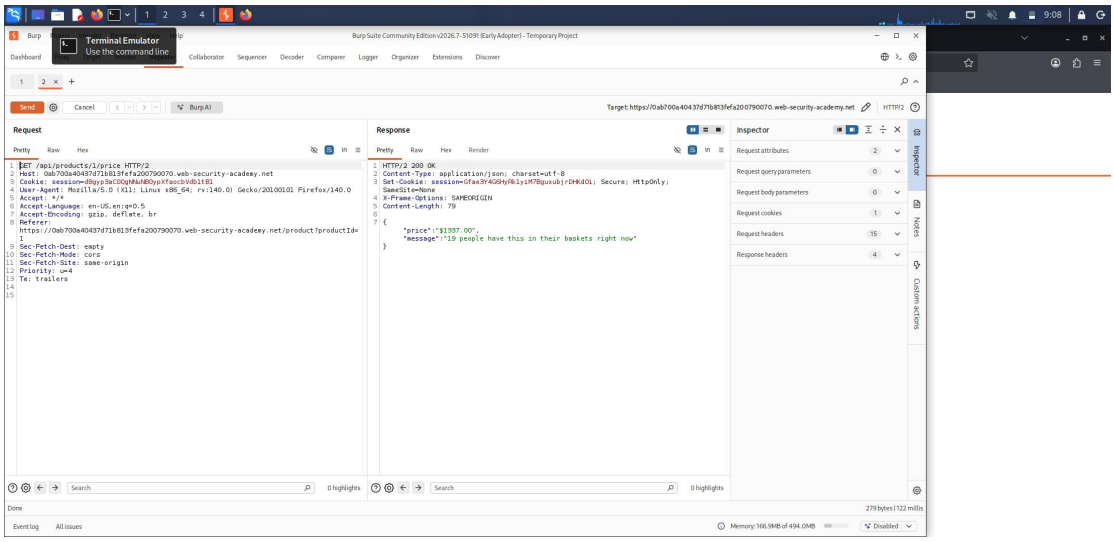


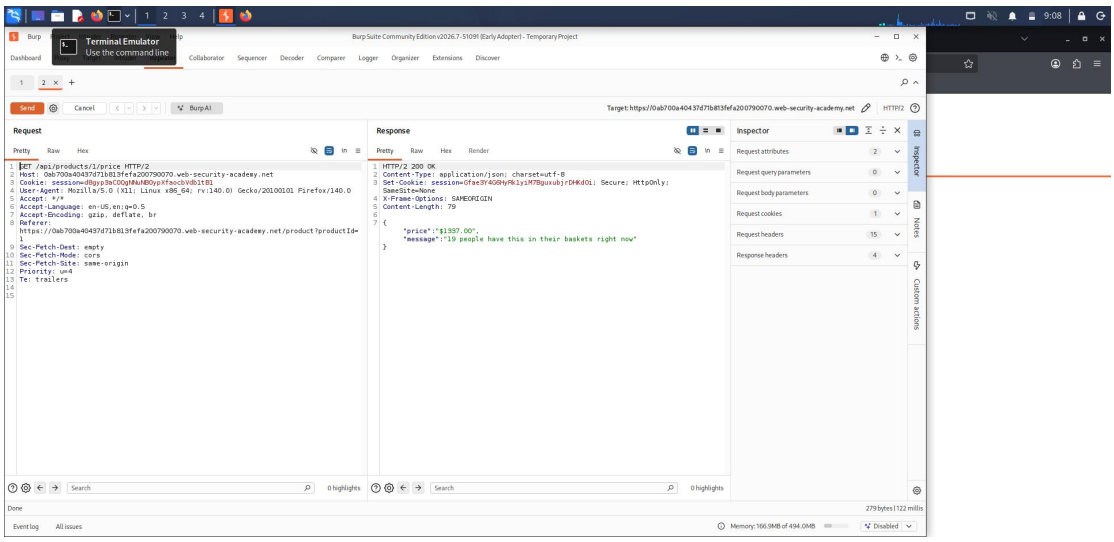


2. I clicked on the **Lightweight "l33t" Leather Jacket** product.



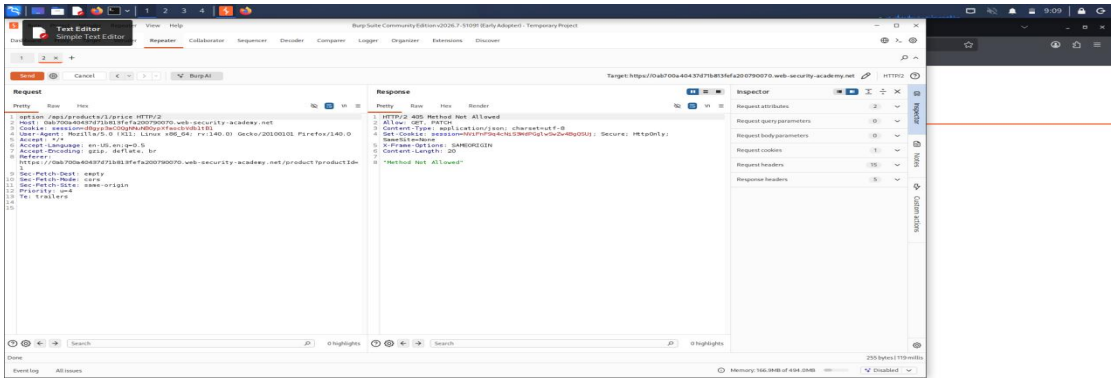
3. In **Proxy > HTTP history**, I right-clicked the ***API/products/1/price*** request for the leather jacket and selected **Send to Repeater**.





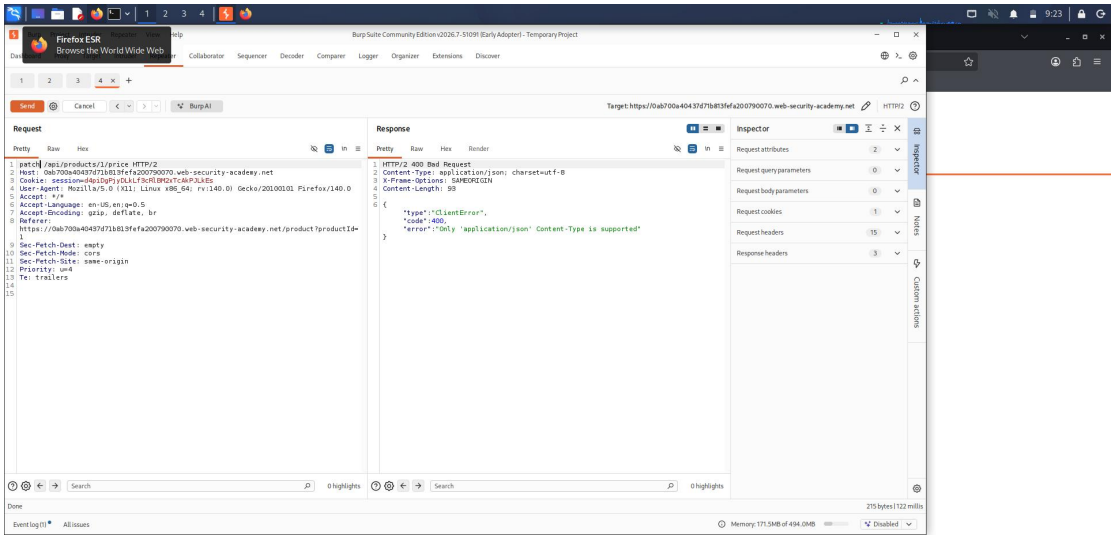
4.In the **Repeater** tab, I changed the HTTP method from GET to OPTIONS and sent the request. The response indicated that the GET and PATCH methods were allowed. This was important because it revealed that the API supported an additional method

that was not being used in the original request. I could then investigate whether the PATCH method allowed unauthorized modification of the product data.

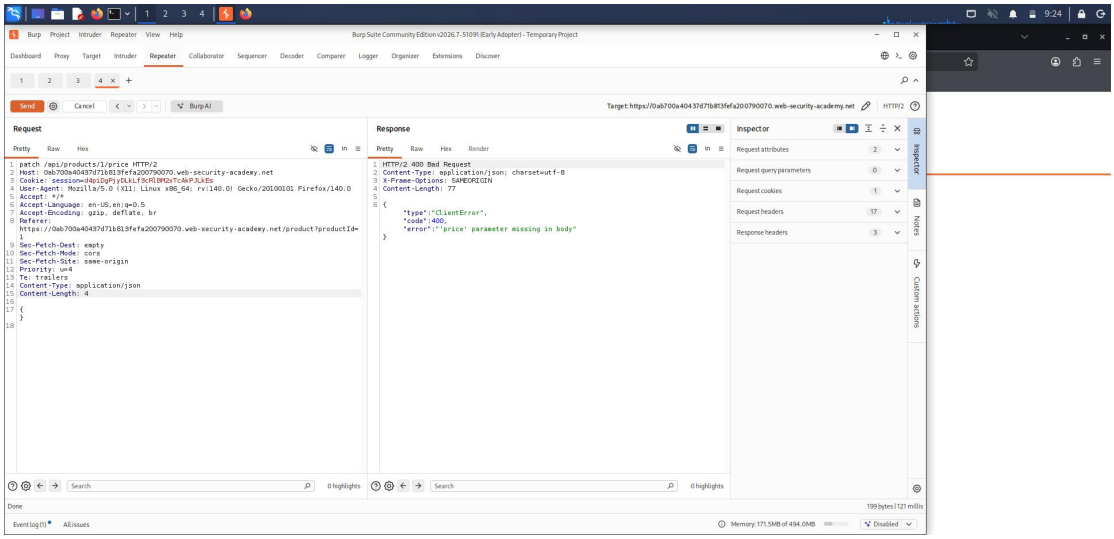


5.In the **Repeater** tab, I changed the method from GET to PATCH, then sent the request. Noticed that this causes an error due to an incorrect Content-Type. The error message specified that the Content-Type should be ***application/json***.

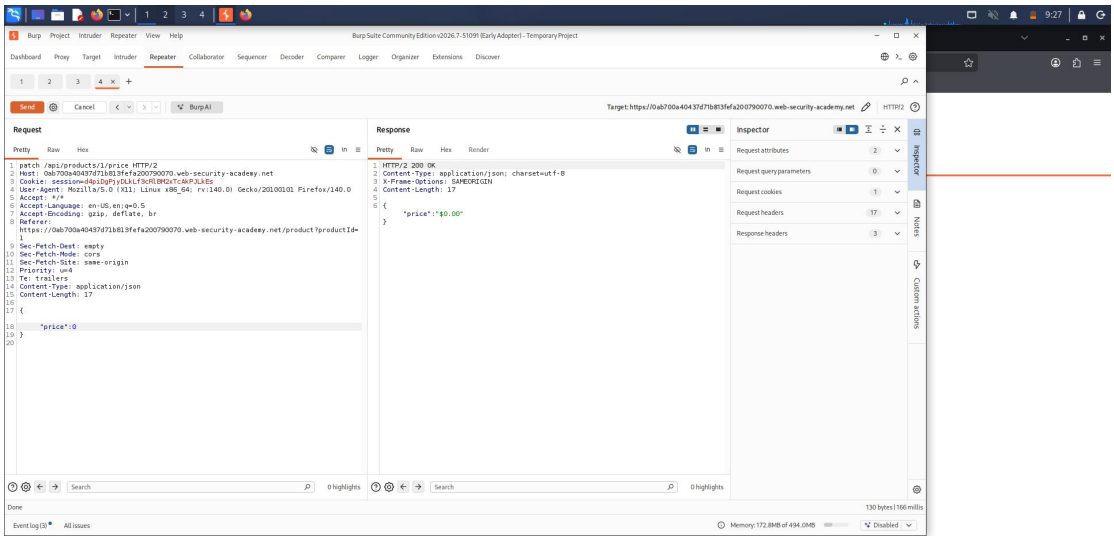
- The error message revealed that the server expected JSON data. I therefore changed the Content-Type header to application/json and sent an empty JSON object. The next error indicated that a price parameter was required. This error-based approach helped me understand the expected structure of the API request.



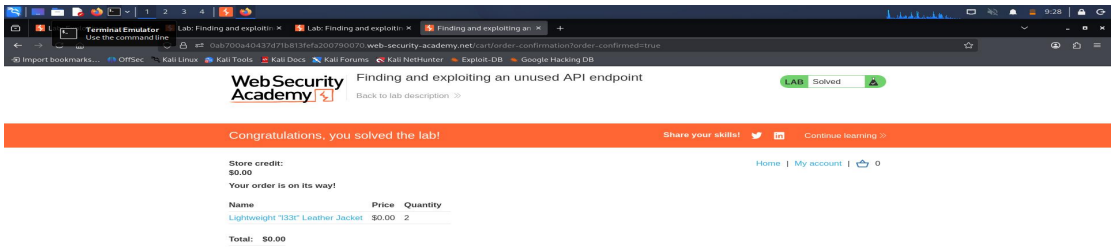
6.I added a Content-Type header and set the value to: ***application/json,*** and an empty JSON object {} as the request body, then sent the request. I Noticed that this causes an error because the request body is missing **a price parameter**.



7.I added a price parameter with a value of 0 to the JSON object: {“price”:0}, and sent the request.



8. In Burp’s browser, I reloaded the leather jacket product page. Noticed that the price of the leather jacket is now **$0.00**.
9. Finally, I added the leather jacket to my basket, and went to my basket, then clicked **Place order** to solve the lab.



- I supplied the price parameter with a value of 0. The product price was subsequently changed to $0.00, demonstrating that an API endpoint that was not directly exposed through the normal website interface could still be accessed and manipulated.

3.LAB3 Exploiting Mass Assignment Vulnerability

https://portswigger.net/web-security/api-testing/lab-exploiting-mass-assignment-vulnerability

❖ **What is a Mass Assignment Vulnerability?**

- A Mass Assignment vulnerability in API testing happens when an API automatically accepts and updates fields that the user should not be allowed to control.

For example

✧ Imagine an API for updating a user's profile.

A normal request might be:

```
{
 "username": "Solomon",
 "email": "solomon@example.com"
}
```

- The application expects users to change only their **username and email**. But suppose the API also has a field:

"isAdmin": false

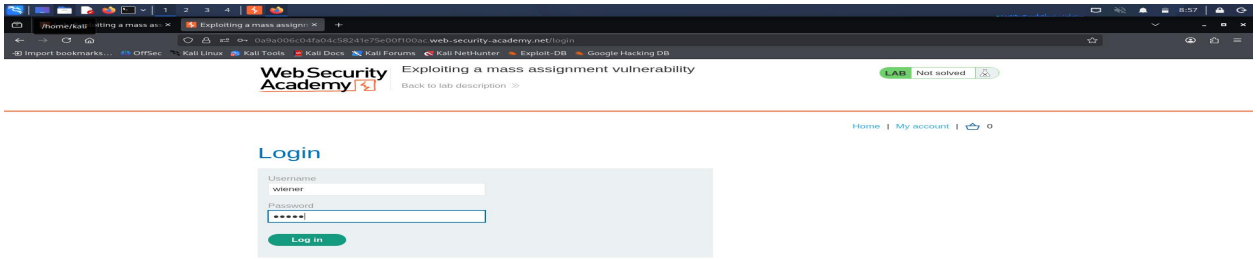
- If the API does not properly restrict which fields can be modified, someone might send:

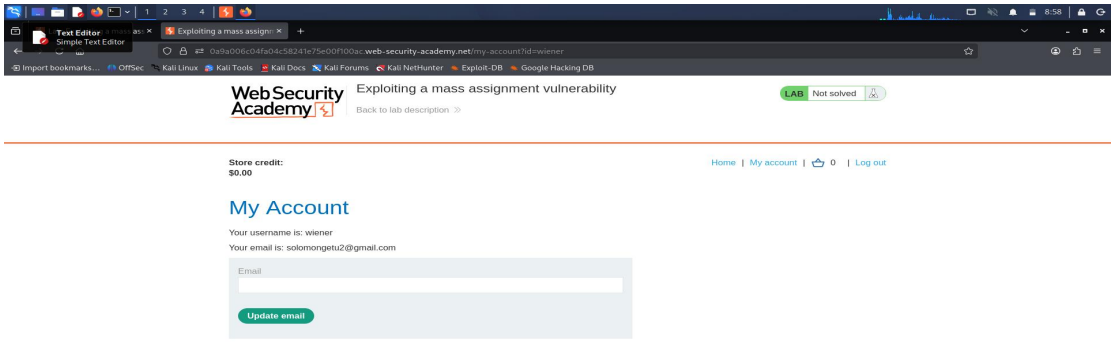
```
{
 "username": "Solomon",
 "email": "solomon@example.com",
 "isAdmin": true
}
```

- If the server accepts and applies isAdmin: true, the user could potentially become an administrator. That's a **Mass Assignment vulnerability**.

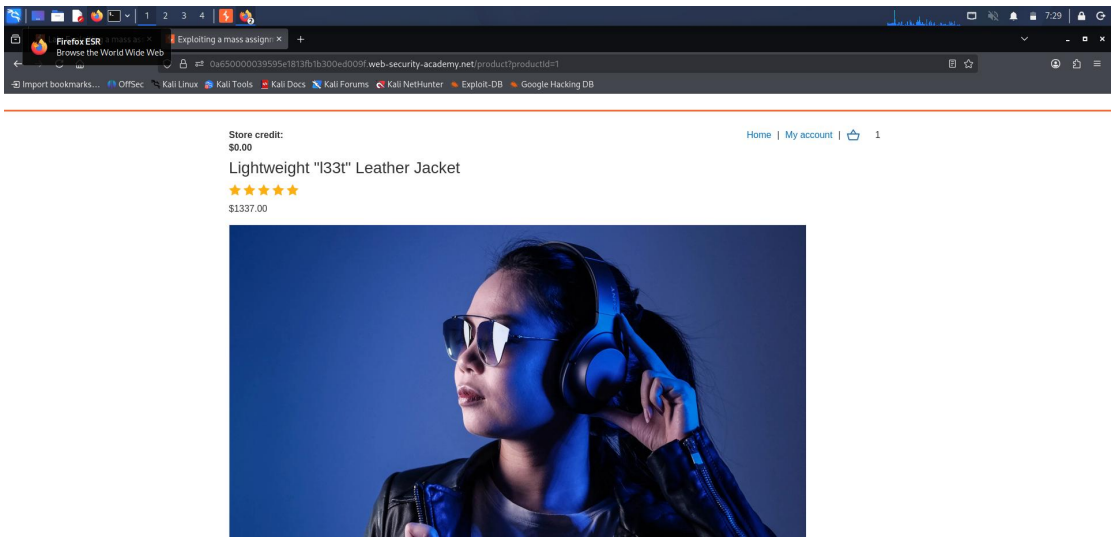
My Walkthrough in Lab3

1. In Burp's browser,I logged in to the application using the credentials wiener:peter, and updated my email.

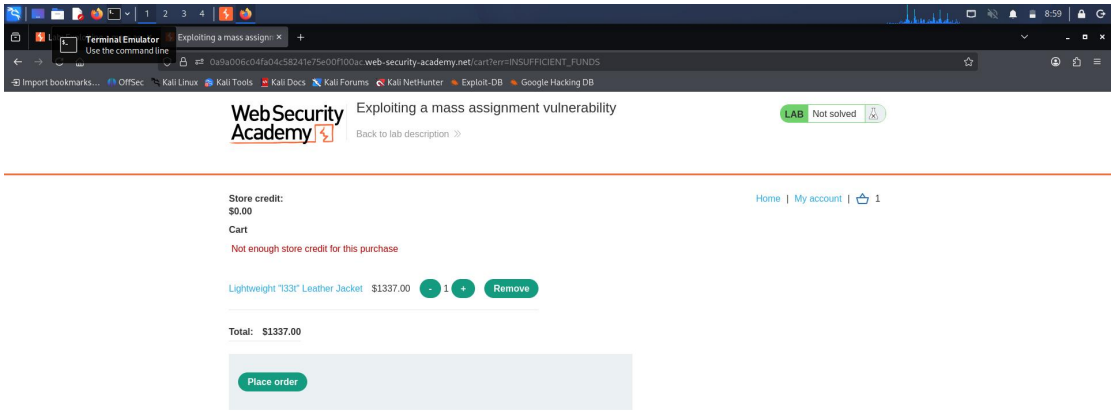




2. I clicked on the **Lightweight "l33t" Leather Jacket** product and added it to my basket.

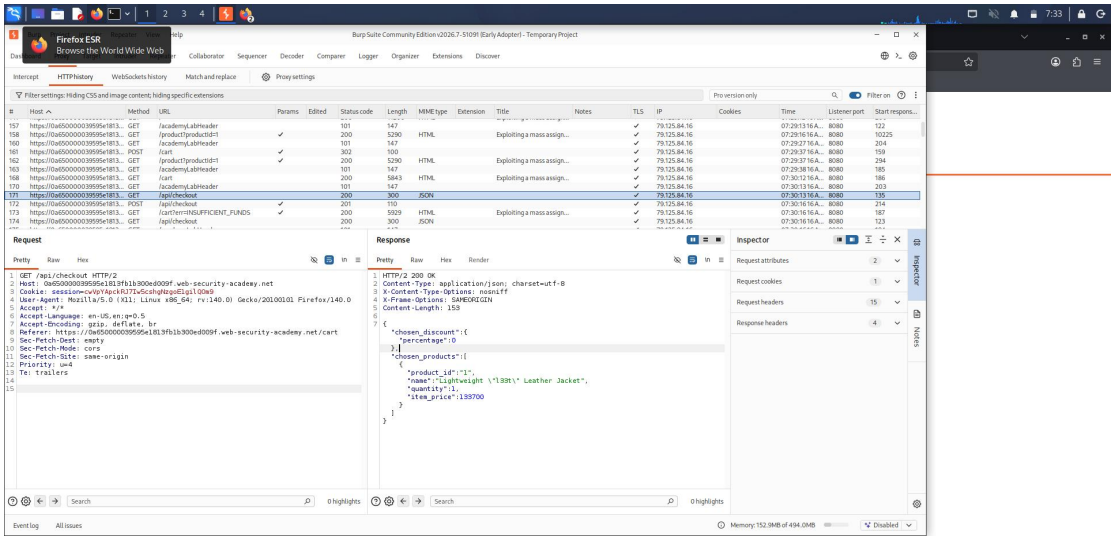


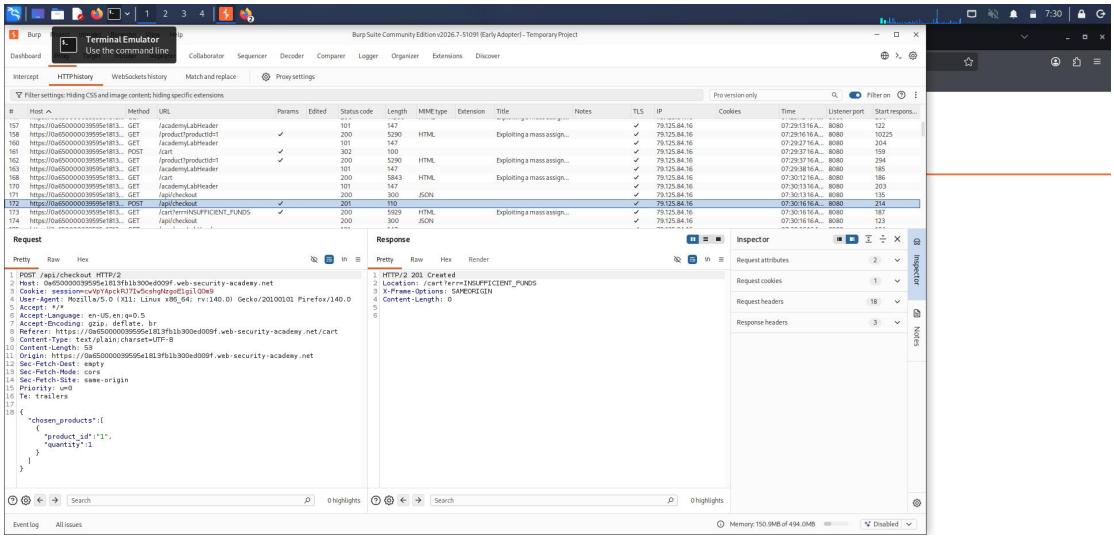
3. I went to my basket and clicked **Place order**.I noticed that I don't have enough credit for the purchase.



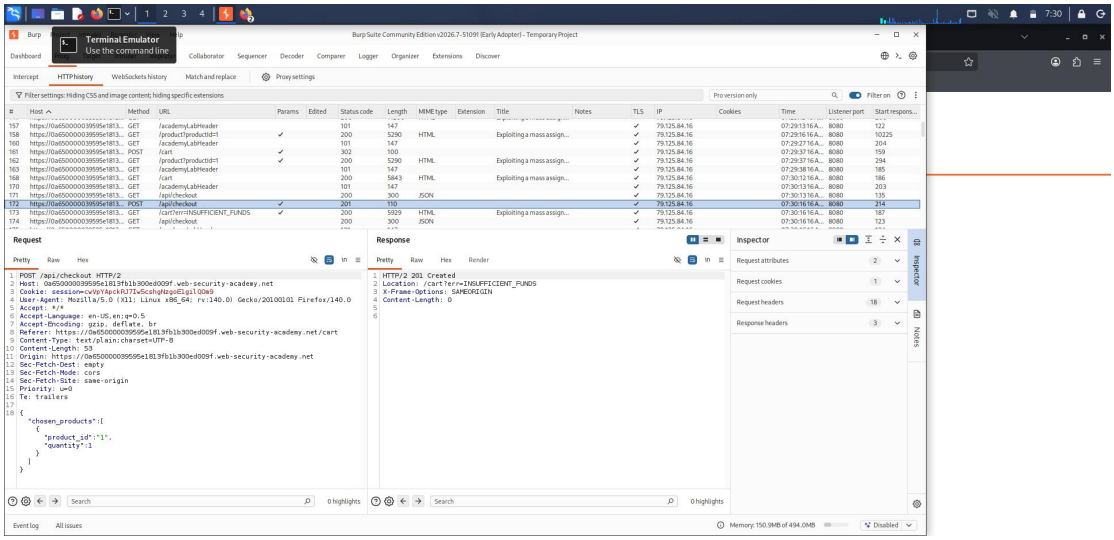
4.I went to **Proxy > HTTP history,** and noticed both the GET and POST API requests for /api/checkout.

5. I checked that the response to the GET request contains the same JSON structure as the POST request. I observed that the JSON structure in the GET response includes a chosen_discount parameter, which is not present in the POST request. This was significant because the chosen_discount parameter was returned by the server but was not included in the normal checkout request. This suggested that the application might accept the parameter even though it was not intended to be controlled by the user.

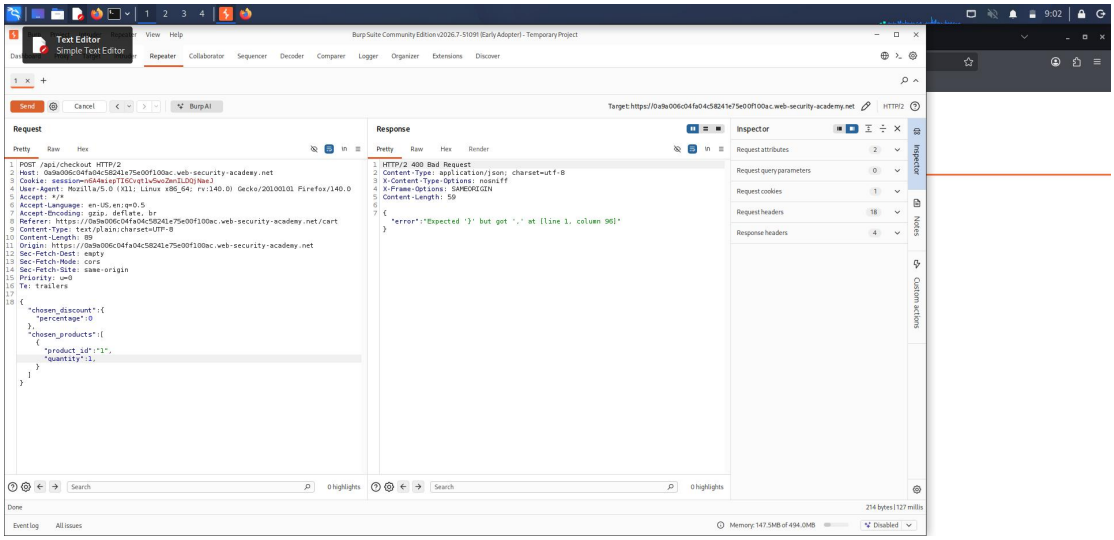




6. I right-clicked the **POST /api/checkout** request and selected **Send to Repeater**.

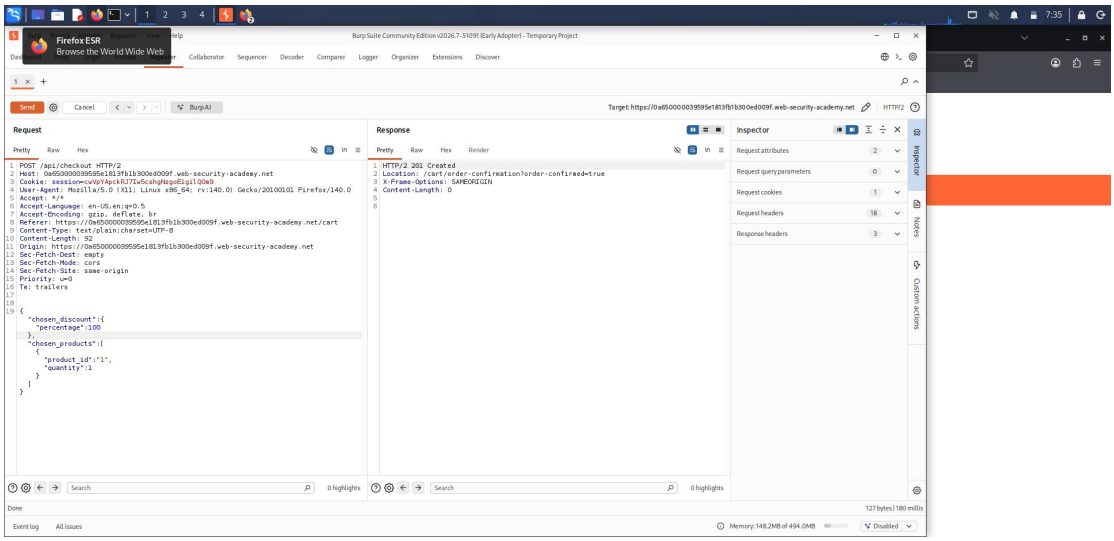


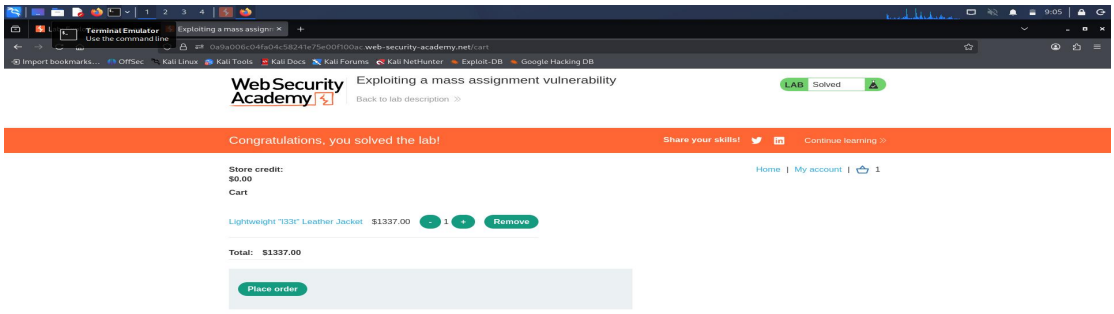
7. In **Repeater**, I added the chosen_discount parameter to the request. The JSON looked like this:



8. I sent the request. Noticed that adding the **chosen_discount parameter** doesn't cause an error.

9. Finally, I changed the chosen_discount percentage to 100, then sent the request to solve the lab.





- I added the chosen_discount parameter to the POST request and observed that the server accepted it without returning an error. I then changed its value to 100. This demonstrated that the application allowed a client-controlled parameter to influence a sensitive checkout value, which is an example of mass assignment.

Conclusion:

In conclusion, these API testing labs helped me understand how APIs can be tested to identify security weaknesses that may not be visible through the normal website interface. Through the labs, I learned how API documentation can reveal available endpoints, how unused API endpoints can potentially be accessed and manipulated, and how mass assignment vulnerabilities can allow unintended parameters to be modified. I also gained practical experience using Burp Suite, particularly Proxy, HTTP History, and Repeater, to inspect and modify API requests and responses. Overall, these exercises improved my understanding of API security and showed me the importance of properly securing API endpoints, HTTP methods, parameters, and access controls.